SESSION 04 / PRINTABLE GUIDE

Session 04: Importable learning notebooks

Complete mandatory pre-delivery setup before class. The presenter must check completion with the customer 3-5 business days before delivery and reconfirm on the preceding business day. Import/attachment/access checks belong to that preparation, not the first minutes of class.

Choose the notebook

Sequence	Participant	Presenter/recovery	Purpose
1. Warm-up	Notebook 101 starter	Completed solution	State, Python basics, Spark, Spark SQL, safe persistence; three short practice cells
2. Repair	Troubleshooting starter	Completed solution	Three deliberate, read-only mistakes: missing variable, column typo, path typo
3. Main lab	Realistic lab starter	Completed solution	Complete scenario pipeline with three short practice checkpoints
4. Independent challenge	Challenge starter	Completed solution	Read-only analysis of a different device/two dates; two code tasks and an interpretation

The package contains **eight notebooks: four starter/solution pairs**. Starters are scaffolded, not empty notebooks. In warm-up/main/challenge starters, `NotImplementedError` marks a practice cell to complete. Replace that cell with your answer; the adjacent Markdown describes the task. Solutions fill those cells but preserve the same safety controls and core transformations. The repair starter instead has three actual, labeled bugs; do not start it with Run all before understanding those intentional failures.

Import and prepare before delivery

1. Download/extract the supplied Session 04 snapshot, keeping the matching handouts and datasets. The v1.0.0 Session 03 release does not contain these.
2. Use the workspace's notebook import action to import the desired `.ipynb`. Import starter and solution as **different notebook items** if both are needed; do not overwrite a participant's work.

3. Confirm **PySpark** is selected and attach the assigned default training Lakehouse. The files intentionally contain no customer resource/environment IDs. Do not assume import selects the correct Lakehouse or runtime.
4. Confirm the prerequisites' scenario files, manifest, and answer keys are already in that Lakehouse. Notebook 101 creates its own four in-memory rows; the repair/main notebooks read the pre-staged scenario sources. The challenge reads persisted main-lab tables after that notebook succeeds.
5. Rehearse all four **solutions** in the approved environment, then test the intended starter experience and actual participant access. Raw source and Warehouse copy activities remain separately preconfigured prerequisites.

Keep the Notebook 101 handout and main-lab handout open alongside the imported items. Use the survival guide for path/error/context checks and the challenge handout for its rubric. Relative Markdown links are repository/package links, not Fabric resource links; they might not navigate correctly inside an imported notebook.

Safety and execution

- **Warm-up:** Writes/read-back are skipped by default. After facilitator approval, `ALLOW_TRAINING_WRITE=True` creates only a new learner-named `s04_101_...` table using `errorifexists`. Follow the handout's separate restart/read-back sequence; do not change the mode to bypass an existing target.
- **Repair:** No writes or deletes. Work through the error cells individually, explain each fix, then restart your own session and run the repaired version from the top. An unexpected setup/access failure is not an exercise bug.
- **Main lab:** `RUN_TRAINING_LAB=False` stops execution before source work or writes in **both** variants. Review the later `s04_lab_` snapshot replacements, verify the default Lakehouse with the facilitator, and only then enable it. Complete practice checkpoints before declaring the notebook successful.
- **Warehouse SQL:** SQL fences in the main handout remain Markdown in the generated notebook. Run publication in the prepared Warehouse as instructed, not as Spark SQL. A successful notebook does not prove copy/Gold publication.
- **Challenge:** Read-only, with two intentional starter practice stops. Complete the main notebook first; no Warehouse publication is required. The separate solution includes the presenter answer key, not learner evidence.

No notebook installs packages, embeds secrets, or carries saved execution outputs. Do not share a later executed copy without reviewing outputs and metadata for sensitive content.

First real-workspace test

Use the supplied matching snapshot and record actual results in an approved customer location. Offline validation does not establish any step below.

1. Complete pre-delivery setup, prove the actual ADLS raw-copy path, and verify participant identity/access and source/answer keys. Record cloud, runtime, assigned workspace/Lakehouse/Warehouse, and snapshot.
2. Import all eight items as separate notebooks, select PySpark, and verify the default Lakehouse and runtime. Record actual import/attachment behavior.
3. Run the warm-up solution with writes off, then complete the starter tasks. Rehearse the optional fresh-target write and restart/read-back sequence only with approval; keep writing off on the read-back rerun.
4. Run repair setup, demonstrate each expected starter error individually, and verify the repaired/solution notebook from a fresh session. Unexpected setup, access, or runtime errors are real blockers, not intentional exercise errors.
5. Review the main solution's default safety stop and overwrite scope. After approval, enable the lab and run in order, including full-data comparisons, charts, and Data Wrangler. Test the starter's three practice stops separately. Record actual source/accepted/rejected/transfer results.
6. Separately execute the prepared Warehouse staging copies and reviewed Warehouse SQL. Verify Gold counts/business results, rerun behavior, and consumer access. A green notebook alone does not complete this step.
7. Attempt the independent challenge, then verify the solution's two daily summaries, zero-rejection day, and explanation after a fresh session. This tests reasoning, not just replaying guided cells.
8. Rehearse Git versioning/review using the separate private practice repository/workspace and read-only notebook copy. Record commit/PR and both sync directions, or label unsupported steps blocked.
9. If included, rehearse VS Code connected authoring: actual cloud sign-in, download, Microsoft Fabric Runtime execution with Fabric run evidence, Publish, Update Notebook, and separate Git commit.

Keep actual run references, discrepancies, resolutions, and skipped/blocked steps. Do not equate imported with executable, published with committed, or presenter success with participant access. No cleanup/reset is automatic.

Maintain and verify

The Notebook 101 and main-lab Markdown are the sources for their worked cells. `<!-- practice: ... -->` markers identify the three answer cells replaced with starter prompts. The three repair exercises and independent challenge cells are defined in the notebook builder; the separate challenge handout supplies its question/rubric.

BASH

```
python lessons/session_04/artifacts/build_notebooks.py
python lessons/session_04/artifacts/validate_examples.py
```

Edit sources and rebuild all eight notebooks, rather than editing only a generated solution. Offline checks cover notebook structure, code syntax, source/solution parity, intentional starter differences, write controls, and local documentation links. They do not prove Fabric import, Spark execution, Data Wrangler, permissions, or Warehouse copy/SQL support. Record those results during the customer rehearsal.

Source: notebooks/README.md. Companion-material links open the repository originals.